

CSC311: Introduction to Machine Learning

Project Report

Submitted by

University of Toronto
Winter 2026
CSC311 Project Report

Data Exploration

At the beginning, the dataset was inspected for completeness and consistency. In this process, rows with more than 50% of their features missing were removed. As a result, this reduced noise and avoided unreliable imputations. For the remaining data, missing numeric and ordinal features were imputed using the median value of the respective feature because these features are skewed in distribution. On the other hand, missing text responses were either left empty or removed if the response contained less than 3 characters, as these provided no meaningful signals.

We then perform exploratory data analysis on the cleaned dataset. Firstly, numeric and ordinal features, including emotional intensity, emotional indicators, number of colours, and number of objects, are shown in Figure 1 below. Notice that most of these features exhibit skewed distributions with indicative separation of values across labels. That is, each label tends to occupy a different cluster of values in each feature, suggesting that the features carry meaningful signals.

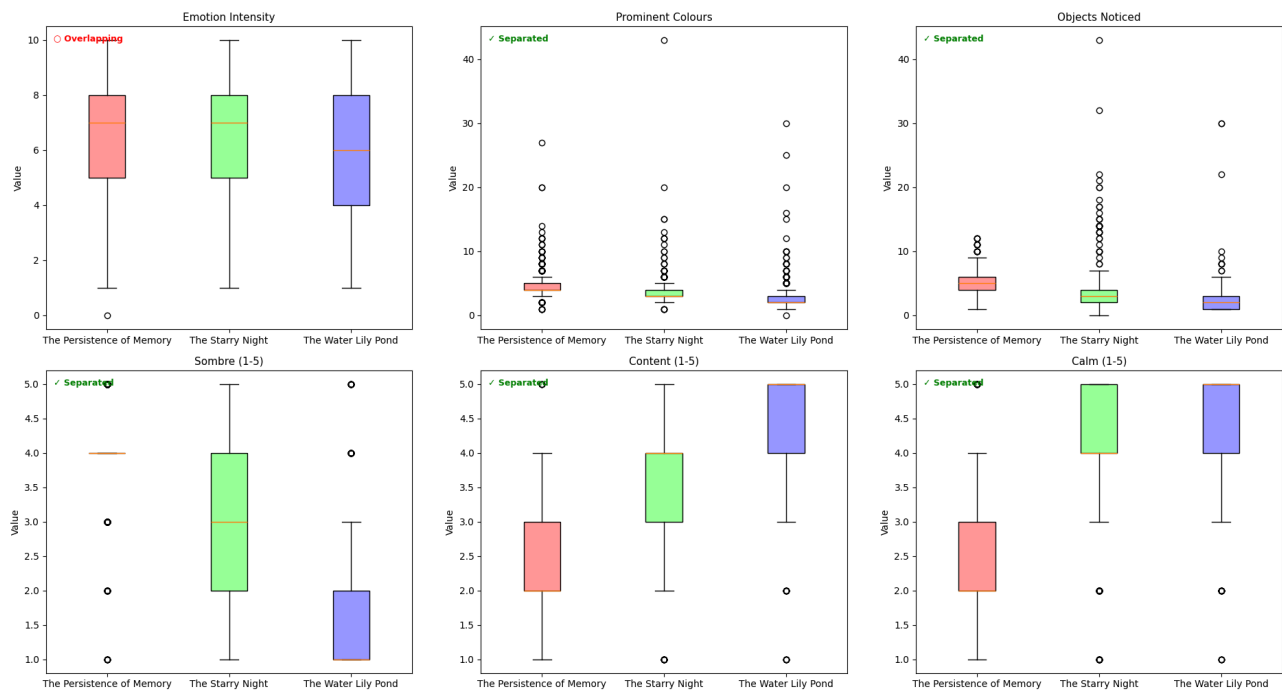


Figure 1: Boxplots of all ordinal features from the machine learning challenge dataset

Note that the boxplot for intensity of emotions shows substantial overlap between labels and is not as clearly separable as the other ordinal features. However, we included all five ordinal features including the feature for intensity of emotions in the model because it still provides some information for “The Water Lily Pond” label. Furthermore, when combined with other ordinal features, it contributes to improved class separation. For all these ordinal features, we represented them as float vectors to allow continuous input into the model.

Next, for categorical features such as sentimental description, viewing preference, food description, and soundtrack description, we used the bag-of-words representation. We converted both multi-class responses and open-ended text responses into one-hot vectors, used as features themselves in the final model, based on the most frequently used words in each feature (excluding prepositions and connective words). Thus, this limits sparsity and preserves common patterns in each label. All categorical features were included in the final model because through performing frequency analysis, we could conclude that certain words appear more often in certain labels, supporting the fact that the bag-of-words representation captures meaningful differences between labels.

Altogether, the multi-layer perceptron model we used to train on the dataset uses all but one feature. This excluded feature is “How much (in Canadian dollars) would you be willing to pay for this painting?”. As shown in Figure 2, this feature is heavily left-skewed with many outliers well beyond the interquartile range of each label. Furthermore, the distribution of values for all three labels are concentrated within the same range, making this feature substantially non-informative for classification and may even add noise to the model.

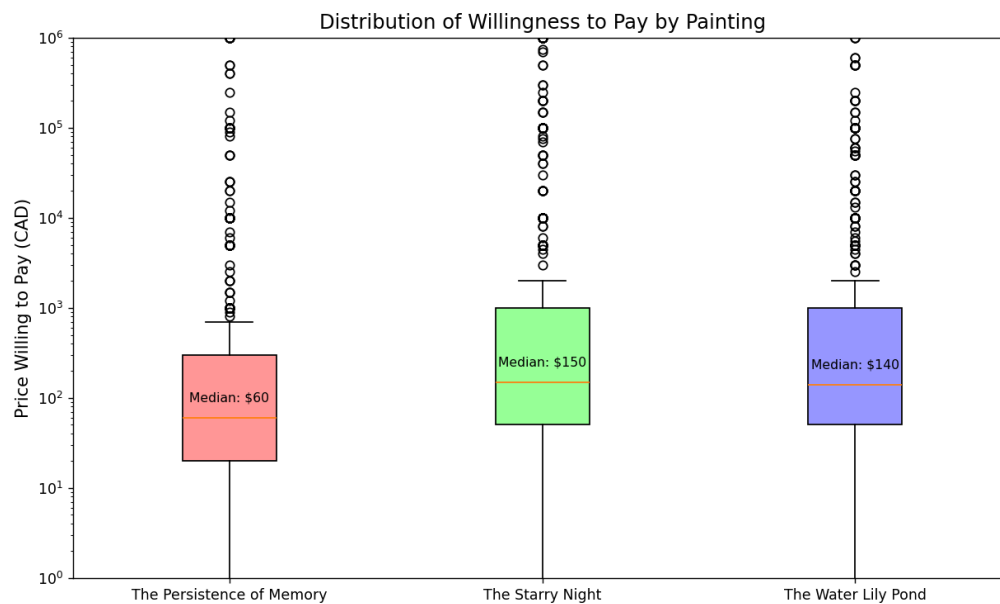


Figure 2: Boxplots showing the distribution of desire to pay for painting feature by label

To prepare the data for training, the dataset is randomly shuffled using a permutation of indices to remove any ordering bias in the dataset. Then, the entire set was split into 80% training and 20% validation sets, preserving overall distribution of data to reliably evaluate the model. Finally, we converted all features into numeric vectors and concatenated them into a singular input matrix for the multi-layer perceptron model to train on.

Model Exploration

We evaluated three model families: a multi-layer perceptron, a random forest, and a naive bayes classifier. Each model was evaluated using the same core features from the dataset that were discussed in the Data Exploration section, with modifications based on model requirements.

Multi-Layer Perceptron: This model was chosen for its ability to learn non-linear relationships between features. All categorical features, including both ordinal responses and open-ended text responses, were encoded using one-hot vectors. For text features, a bag-of-words representation was first applied to extract the most frequently used words, where each selected word was then represented as a one-hot feature. All numeric features were normalized to ensure stable stochastic gradient descent and to ensure that all features carry equal weight. Any missing numeric features in the data were imputed using medians as previously described, while missing text entries were assigned a zero vector after encoding, representing NaN handling on text features for the model.

Random Forest: This model was selected for its ability to handle non-linear relationships, mixed feature types, and outliers. All categorical features were once again, represented as one-hot vectors. However, for numeric features, we do not normalize as the decision tree based model is not affected by scaling of features. On the contrary, we handle missing entries differently. Missing numeric features were still imputed with medians to avoid excess noise during training. However, missing categorical features do not require modification, as the random forest automatically handles missing text entries as zero vectors. Also, compared to the multi-layer perceptron model, the random forest model is much more lenient towards noise and redundant features.

Naive Bayes: Compared to the two previous models, the Naive Bayes model is much simpler and works well on smaller datasets like the machine learning challenge dataset. However, because the model assumes conditional independence between features given the label, we need to adjust the representation of the features to better match the model. We used a variant of the Naive Bayes model called the multinomial Naive Bayes for frequency-based feature analysis. All numeric features were adjusted into vectors of integers, with any floats being rounded while for text features, we directly use the frequency counts analyzed through bag-of-words rather than convert the most frequent words into one-hot vectors. Note that we also had to convert all numeric and ordinal features into binaries to convert them into categorical values so that they could be used in the multinomial model.

Model Choice and Hyperparameters

We evaluated the three model families: a multi-layer perceptron, a random forest, and a Naive Bayes classifier, as outlined in the Model Exploration section. To ensure comparability, all three models were trained and evaluated on the same preprocessed dataset and the same 80% training and 20% validation set split that was shuffled once. Note that no model had access to the validation set during training to ensure that differences in performance are due to model architecture rather than the difference in data. The primary evaluation metric is cross-entropy loss as all the models utilized multiple classes to classify the labels. As a result, cross-entropy loss directly measures how well the predictions of the model match the true labels. Also, we will use validation accuracy as another metric to directly evaluate the performance of the classifier.

Naive Bayes: The Naive Bayes model achieved validation loss between 0.40 and 0.50, with validation accuracy around 64%. Its performance appears limited by the conditional independence assumption, which is clearly violated in the dataset the model uses. See that the bag-of-words representation of text features are often correlated, which reduce the effectiveness of the Naive Bayes model to capture relationships.

Random Forest: The random forest model improves upon model performance significantly, achieving validation loss between 0.17 and 0.31 and validation accuracy between 88% to 95% over multiple runs. Comparatively, this model captures non-linear interactions and handles multiple feature types much better. However, we noticed that training time and memory usage were substantially higher than the other models, making it less practical under time constraints.

Multi-layer Perceptron: The multi-layer perceptron model consistently achieved a validation loss between 0.19 and 0.34, with validation accuracy between 87% to 91% during initial selection. We see that this model's performance is comparable to the random forest model, while requiring significantly less training time and offering more control through tuning more hyperparameters. For these reasons, we ultimately selected the multi-layer perceptron model as the model used in `pred.py`.

For the multi-layer perceptron, we tuned the following hyperparameters: number of hidden perceptrons, learning rate, batch size (for stochastic gradient descent variant of the model), and number of epochs. Note that the activation function was fixed to ReLU.

To search for the most optimal hyperparameters, we followed a structured search over a small batch of values for each hyperparameter, fixing all other hyperparameters. Through a quick test, we set the initial hidden perceptron to 1500, as training the model above 5,000 perceptrons takes a long time and returns unsubstantial results, while perceptron values lower than 500 vary greatly in both validation accuracy and loss across runs. The initial batch size will be set to 100 as smaller batch sizes introduce greater variability when performing stochastic gradient descent, and larger batch sizes

on the dataset makes the stochastic gradient descent behave as batch gradient descent. Similarly, the initial number of epochs will be set to 200. A smaller amount of epochs raises the loss and lowers the accuracy as the model is not given enough time to converge, while a larger amount of epochs leads to overfitting the training set and increases validation loss.

We begin by varying the learning rate, as it is the most sensitive to change. Through some quick experiments, we set an initial learning rate of 0.1. The validation accuracy results in about 86% while the validation loss is 0.27. At a learning rate of 0.5, the validation accuracy is about 35% while the loss is 1.13. This implies that the model diverges and is unstable at such a high learning rate, so instead, we decrease the learning rate. Thus, by decreasing the learning rate to 0.03, the final validation accuracy is about 91%, with a loss of 0.26, a remarkable increase in performance. Decreasing the learning rate further to 0.01 yields a validation accuracy of 73% and a validation loss of 0.39. That is, below a learning rate of 0.03, the model begins to underfit. Therefore, the best learning rate is at around 0.03.

Next, we choose the batch size that would perform the best for the multi-layer perceptron model. We first tested a batch size of 75. The resulting validation loss yields a value of 0.27 and the validation accuracy is about 75%. Thus, we can pick a batch value between 75 and 100, say 90. At this hyperparameter value, the resulting validation loss is about 0.24 and the accuracy is about 85%. Since we only slightly decreased the batch value, we can test a slightly increased batch value instead. A model tested on a batch value of 110 yields a validation accuracy of 89% and a validation loss of 0.25, which is a slight decrease compared to using a batch size of 100, so we keep the batch size at 100 as it provides stable performance with low variance across runs.

Then, we can choose the number of hidden perceptrons for the model. We first test by decreasing the number of perceptrons to 500. The validation accuracy from this test is about 88% and results in a validation accuracy of about 0.24. So instead, we can try increasing the number of hidden perceptrons used. For example, we can test 2500 hidden perceptrons. However, at 2500 hidden perceptrons, the validation accuracy decreases to about 87% and the loss increases to about 0.24. Furthermore, the file size increases to over 10 MB. Therefore, we can continue to use our initial number of hidden perceptrons at 1500 as it appears to be around where the model performs the best while being under 10 MB.

Lastly, we tune the number of epochs. By decreasing the number of epochs to 100, we see the model performance degrade as the validation accuracy decreases to about 72% and the validation loss increases to 0.34. Hence, this indicates that the model is terminating before it converges. We continue and try 400 epochs. At 400 epochs, we notice that the validation accuracy increases to 93% and the loss remains rather similar to the values reported at 200 epochs. As a result, it shows that at around 400 epochs, the model converges without clear overfitting. Further increasing the number of epochs increases the training accuracy to 100%, but the validation accuracy decreases. Not only is the cost of training time significantly increased, but the model also overfits tremendously.

Therefore, using structured searching and some initial observations about different hyperparameter values, we conclude that the best observed configurations of hyperparameters for the final model is when the hidden perceptrons is set to 1500, the learning rate is set to 0.03, the batch size is set to 100, and the number of epochs is set to 400. These hyperparameters not only consistently produced the lowest validation loss and the highest validation accuracy among the tested combinations of hyperparameters above, but also maintained reasonable training time and size of the model.

Prediction

We expect the model to perform with an accuracy of 93% on the test set. This estimate is based on consistent validation performance, a small generalization gap, and balanced performance across classes.

Our final model uses 1500 hidden perceptrons, a learning rate of 0.03, a batch size of 100, and 400 epochs, as discussed in the Model Choice and Hyperparameter section.

To evaluate the stability of the model, we trained the model 5 times with different random initializations as well as random shuffles between the training set and the validation set. The validation accuracies across these 5 runs were 0.927, 0.920, 0.934, 0.941, and 0.922. The mean validation accuracy across these runs is 0.929, with all results clustered in a range of about 2%. As a result, this indicates stable model performance across different initializations when the training and validation set have slight modifications to their data.

We also compare training and validation accuracy to assess overfitting. We see that training accuracy for the five runs were 0.980, 0.950, 0.970, 0.970, and 0.980. This results in a generalization gap between 0.009 to 0.060. For a multi-layer perceptron model with 1500 hidden perceptrons, this gap is small, so it indicates that the model has limited overfitting. To further support this conclusion, the validation loss for the 5 runs are 0.286, 0.182, 0.257, 0.177, 0.259, which remain relatively low. Altogether, these results suggest that the multi-layer perceptron model generalizes well.

To further evaluate performance, we examined the confusion matrices for each run. Each class achieved accuracy between 0.864 and 0.973, showing that the performance is balanced across classes. That is, the model is not biased towards any singular label and implies that the overall validation accuracy is a reliable metric to be used.

Through these evaluations, we expect the model to achieve approximately 93% accuracy on the test set.